

A Convolutional Neural Network classifier of the CIFAR-10 Dataset

Group 19: James Burnell, Ivan Barinskiy, Owen Crucefix, Michael Harrison, Scott Redshaw

Abstract—This paper presents a CNN architecture building on the AlexNet and VGG block concepts. It is applied to the CIFAR-10 dataset hosted by the University of Toronto. Over the course of 15 training epochs, the proposed architecture achieves an accuracy of 83.59% over the dataset's official testing portion, or an error rate of 16.41%.

I. INTRODUCTION

The task for this group project is to create a neural network classifier that is used to classify different images into separate categories depending on the image. From using convolution neural networks, we will create an image classification algorithm, which uses the CIFAR-10[1] data set to classify the images into 10 different categories.

At the start of image classification, the programmers needed to extract meaningful and low dimensional features that could be worked on, which led to the creation of convolution neural networks known as the AlexNet[2]. Through many iterations and a lot of research, googLeNet was created which takes inputs with different sized convolutions and takes advantage of multi-level feature extraction[3]. GoogLeNet is used in computer vision tasks like face recognition and detection. AlexNet achieved top-1 and top-5 test set error rates of 37.5% and 17.0% whereas googLeNet has an error rate of 6.67% [4]. Image classification is a very important part of digital analysis, and the use of machine learning can increase the speed, accuracy and number of images classified compared to human classification.

II. METHOD

The network we designed builds on an existing architecture called AlexNet as well as the concept of a VGG block and is implemented in PyTorch. We have four blocks of ' $C \rightarrow C \rightarrow MP \rightarrow BN$ ' where C is a convolutional layer with padding, MP is a Maxpool layer with stride of 2 (halving the output's resolution as a result), and BN is a batch normalization layer. The blocks feed from one to the next. After the last block, we flatten the result (which has the shape (512,2,2)) and feed it to three fully connected layers which go from $512 * 2 * 2 \rightarrow 1024 \rightarrow 512 \rightarrow 10$. The convolutional and linear layers use the ReLu activation function which returns 0 if the result is negative, or the input value if it was positive. To attempt to combat overfitting, we included dropout layers between each of the linear layers and used a dropout probability of 0.6. The overall architecture was deemed fitting for the task as convolutional neural networks are frequently used to classify images, with an increased depth allowing for more parameters to make the network more generalized.

Batch normalization was also important for the network's performance as it ensured that all inputs to the blocks were on the same scale, as well as allowing for a greater learning rate, increasing the rate at which the model is able to fit to the data.[5]

For the loss function, we used the cross-entropy loss function as shown below:

$$H(X) = - \sum_x p(x) \log(q(x))$$

where x is a class in X , $p(x)$ is the true probability distribution (one-hot), and $q(x)$ is the model's predicted probability distribution.

Before being passed into the network, the images were converted to tensors and normalized using each colour channel's mean and standard deviation. The values for the mean and standard deviation were calculated in a separate python script available in the repository.

III. DIAGRAM

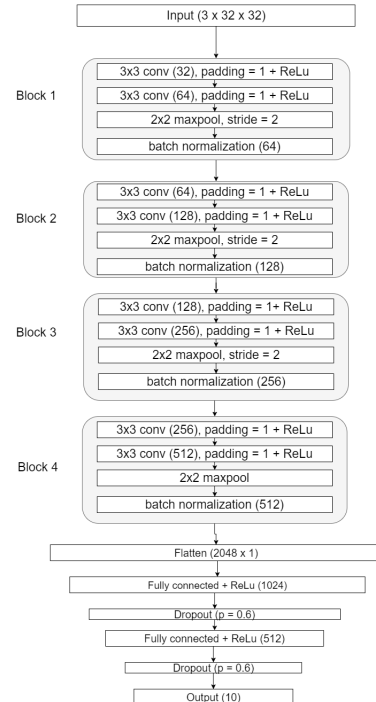


Fig. 1: Network architecture visualisation

IV. RESULTS AND EVALUATION

Class	Accuracy (%)
Plane	86.1
Car	90.6
Bird	75.6
Cat	71.6
Deer	79.9
Dog	74.8
Frog	88.6
Horse	87.6
Ship	91.3
Truck	89.8
Mean	83.59

TABLE I: Success rates for different image subjects

In order to evaluate the performance of the proposed model, accuracy was chosen as the preferred metric. This choice was justified not only as a response to the project requirement (a sub - 30% error rate) but also due to the perfectly equal number of images belonging to each class. This meant that accuracy provided a clear and unbiased look at the model's performance. The model utilizing the proposed architecture was able to correctly classify the images at least 70% of the time with some of the categories reaching as high as a 91% (rounded) success rate, as shown in Table I. The overall success rate of the model was 83.59% when tested using the 10000 test images from the CIFAR-10 data set. We calculated the test accuracy of each class by running the model on the test data. The softMax activation function is then applied to the outputs giving us a list of probabilities that specify how likely the model thinks that each image belongs to each class, taking the largest probability for each image as the model's prediction. We then go through and check if each prediction the model gave was correct. We then calculated the success rate of the model overall, and each class individually, as a percentage of the correct predictions that the model gave. Once the accuracy was calculated we subtracted it from 100 to get the classification test error.

For the optimization function, we used a stochastic gradient descent with a learning rate of 0.001, chosen after trying several values, and a momentum of 0.9. The momentum value used is the "recommended" value, often referenced in the machine learning community[6].

For training we ran the model for 15 epochs over the training data calculating the accuracy each time to measure its progress over the epochs. The loss for the current weights is then calculated using the cross-entropy loss function and the model is adjusted for the next epoch. The change in the value produced from the loss function over the training period can be seen in Fig. 2 and the change in the test set accuracy in Fig. 3. The network was trained in a Jupyter environment on a Nvidia GTX 1050 (Mobile) with 2GB of GDDR5 VRAM and 16GB of LPDDR3 1866MHz system memory resulting in a training time of 31 minutes and 41 seconds.

V. CONCLUSION

Overall, the classifier achieved the accuracy requirement of 70%, being able to classify images with a success rate of 83.59%. The chosen architecture was able to achieve this

accuracy with a relatively short training time which made it the ideal architecture given the relatively short timeframe of the project as well as the team's hardware limitations. A potential area for improvement of this project is the model's tendency to overfit, which is evident in the extremely low training loss seen on Fig. 2 corresponding to a small increase in testing loss in the later epochs seen on Fig. 2. This could be achieved by augmenting the dataset with more images or inserting dropout layers within each of the blocks, however, the benefit of dropout in convolution layers remains an area of active research[7].

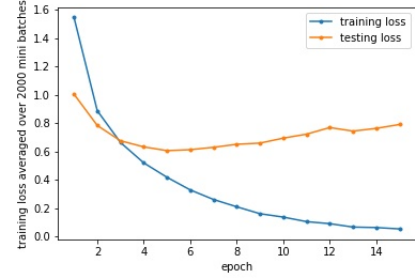


Fig. 2: Training loss over the epochs

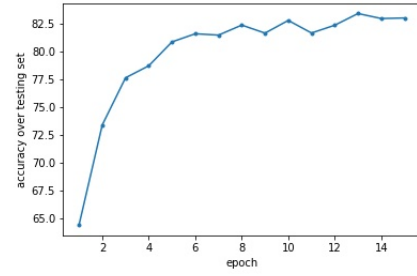


Fig. 3: Accuracy (%) over the test set sampled every training epoch

REFERENCES

- [1] A. Krizhevsky, 'Learning Multiple Layers of Features from Tiny Images', University of Toronto, 2009[Online]. Available: <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>.
- [2] A. Krizhevsky, I. Sutskever and G. Hinton, "ImageNet classification with deep convolutional neural networks", Communications of the ACM, vol. 60, no. 6, pp. 84-90, 2017. Available: 10.1145/3065386.
- [3] C. Szegedy et al., "Going deeper with convolutions," 2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2015, pp. 1-9, doi: 10.1109/CVPR.2015.7298594.
- [4] "A Guide to AlexNet, VGG16, and GoogleNet — Paperspace Blog", Paperspace Blog, 2022. [Online]. Available: <https://blog.paperspace.com/popular-deep-learning-architectures-alexnet-vgg-googlenet/>. [Accessed: 10 May 2022].
- [5] M. Chablani, "Batch Normalization", Towards data science, 2017. [Online]. Available: <https://towardsdatascience.com/batch-normalization-8a2e585775c9>. [Accessed: 10- May- 2022].
- [6] V. Bushaev, "Stochastic Gradient Descent with momentum", Towards data science, 2017. [Online]. Available: <https://towardsdatascience.com/stochastic-gradient-descent-with-momentum-a84097641a5d>. [Accessed: 10- May- 2022].
- [7] J. Tompson, R. Goroshin, A. Jain, Y. LeCun and C. Bregler, "Efficient object localization using Convolutional Networks," 2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2015, pp. 648-656, doi: 10.1109/CVPR.2015.7298664.